

Material de apoyo para la
realización de la P3 de TSI

Errores típicos y mensajes asociados

Nota: se proporcionan mensajes asociados a Fast Downward (FD) y Fast Forward (FF). FF fue el planner empleado hasta el curso 2023-24. FD es el planner que se debe emplear este curso (junto con el alias de ejecución seq-opt-lmcut; salvo en el ejercicio 4 en donde se debe emplear seq-sat-lama-2011).

Errores típicos y mensajes asociados (1)

- Falta un paréntesis:

- FD:

Error: Could not parse domain file: dominio1.pddl  ¡No indica en qué línea!
Reason: Missing ')'

- FF:

dominio1.pddl: syntax error in line 46, '(':  Te indica la línea.
domain definition expected ¡Pero no es en la que falta el paréntesis, sino una de las siguientes!

Aunque si falta, por ejemplo, el de fin de fichero, el error es más críptico:

dominio1.pddl: syntax error in line 68, “:  ¡No indica que falta un paréntesis!
domain definition expected

Errores típicos y mensajes asociados (2)

- En los parámetros de una acción empleamos tipos erróneos (por ejemplo `:parameters (?per - Personaje ?x ?y - Localizacion2)` en lugar de `:parameters (?per - Personaje ?x ?y - Localizacion):`

- FD:

Search stopped without finding a solution.

No da error! Sencillamente dice que no encuentra un plan!



- FF:

warning: parameter ?X of op VIAJAR has unknown or empty type LOCALIZACION2.
skipping op

ff: goal can be simplified to FALSE. No plan will solve it

Aunque el mensaje no es muy claro (ni siquiera indica en qué línea ocurre), sí que menciona explícitamente el tipo inexistente!



Errores típicos y mensajes asociados (3)

- En los parámetros de una acción empleamos una variable que, luego, en el cuerpo de la acción no se llama igual. Por ejemplo, `:parameters (?per - Personaje ?w ?y - Localizacion)`, cuando internamente empleamos `?x` e `?y`:

- FD:

Search stopped without finding a solution.

No da error! Sencillamente dice que no encuentra un plan!

- FF:

undeclared variable ?X in literal EN. check input files

Este sería el primer predicado en que se emplea la variable ?x.

Aunque el mensaje no es muy claro (ni siquiera indica en qué línea ocurre), sí que menciona explícitamente la variable conflictiva!

Errores típicos y mensajes asociados (4)

- En el fichero de problema, en lugar de declarar Enano1 y Hobbit1 de tipo Personaje, los declaramos como de tipo Enano y Hobbit (y estos últimos no son tipos, sino que son constantes):

- FD:

```
Generating Datalog program... b'Traceback (most recent call last):\n File\n"/mnt/c/Users/pablo/Desktop/TSI_2024-25/P3/Soluciones/FD/fast-downward-\n24.06.1/builds/release/bin/translate/translate.py", line 747, in <module>\n  main()\n File\n"/mnt/c/Users/pablo/Desktop/TSI_2024-25/P3/Soluciones/FD/fast-downward-\n24.06.1/builds/release/bin/translate/translate.py", line 718, in main\n  sas_task =\n  pddl_to_sas(task)\n File "/mnt/c/Users/pablo/Desktop/TSI_2024-25/P3/Soluciones/FD/fast-\ndownward-24.06.1/builds/release/bin/translate/translate.py", line 547, in pddl_to_sas\n  reachable_action_params) = instantiate.explore(task)\n File "/mnt/c/Users/pablo/Desktop/TSI_2024-\n25/P3/Soluciones/FD/fast-downward-24.06.1/builds/release/bin/translate/instantiate.py", line 116, in\n  explore\n  prog = pddl_to_prolog.translate(task)\n File "/mnt/c/Users/pablo/Desktop/TSI_2024-\n25/P3/Soluciones/FD/fast-downward-24.06.1/builds/release/bin/translate/pddl_to_prolog.py", line\n  167, in translate\n  translate_facts(prog, task)\n File "/mnt/c/Users/pablo/Desktop/TSI_2024-\n25/P3/Soluciones/FD/fast-downward-24.06.1/builds/release/bin/translate/pddl_to_prolog.py", line\n  153, in translate_facts\n  translate_typed_object(prog, obj, type_dict)\n File\n"/mnt/c/Users/pablo/Desktop/TSI_2024-25/P3/Soluciones/FD/fast-downward-\n24.06.1/builds/release/bin/translate/pddl_to_prolog.py", line 146, in translate_typed_object\n  supertypes = type_dict[obj.type_name].supertype_names\nKeyError: 'enano'\n\ntranslate exit code: 30
```

Driver aborting after translate

Da un error muy verboso y poco informativo. Aunque, al final de todo, sí nos da una pista con el término que origina el error.

Errores típicos y mensajes asociados (4)

- En el fichero de problema, en lugar de declarar Enano1 y Hobbit1 de tipo Personaje, los declaramos como de tipo Enano y Hobbit (y estos últimos no son tipos, sino que son constantes):

- FF:

predicate PERSONAJEES is declared to use unknown or empty type PERSONAJE



Aquí lo que nos viene a decir es lo siguiente:

- Tienes un predicado definido como `(personajeEs ?per - Personaje ?tPer - tipoPersonaje)`
- Pero lo utilizas con `(personajeEs ?per - tipoPersonaje ?tPer - tipoPersonaje)`, dado que Enano y Hobbit son constantes de tipo `tipoPersonaje`
- Es decir, el primer parámetro debería ser `Personaje`, pero no es eso lo que recibo...

Errores típicos y mensajes asociados (5)

- En el fichero de dominio, falta un espacio a la hora de definir un tipo. Por ejemplo, tenemos **Recurso- Ubicable** en lugar de **Recurso - Ubicable**:

- FD:

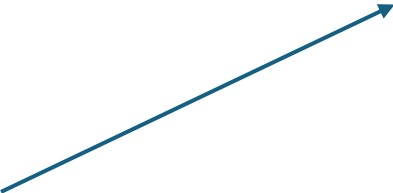
```
Generating Datalog program... b'Traceback (most recent call last):\n File\n"/mnt/c/Users/pablo/Desktop/TSI_2024-25/P3/Soluciones/FD/fast-downward-\n24.06.1/builds/release/bin/translate/translate.py", line 747, in <module>\n  main()\n File\n"/mnt/c/Users/pablo/Desktop/TSI_2024-25/P3/Soluciones/FD/fast-downward-\n24.06.1/builds/release/bin/translate/translate.py", line 718, in main\n  sas_task =\n  pddl_to_sas(task)\n File "/mnt/c/Users/pablo/Desktop/TSI_2024-25/P3/Soluciones/FD/fast-\nfast-downward-24.06.1/builds/release/bin/translate/translate.py", line 547, in pddl_to_sas\n  reachable_action_params) = instantiate.explore(task)\n File "/mnt/c/Users/pablo/Desktop/TSI_2024-\n25/P3/Soluciones/FD/fast-downward-24.06.1/builds/release/bin/translate/instantiate.py", line 116, in\n  explore\n  prog = pddl_to_prolog.translate(task)\n File "/mnt/c/Users/pablo/Desktop/TSI_2024-\n25/P3/Soluciones/FD/fast-downward-24.06.1/builds/release/bin/translate/pddl_to_prolog.py", line\n167, in translate\n  translate_facts(prog, task)\n File "/mnt/c/Users/pablo/Desktop/TSI_2024-\n25/P3/Soluciones/FD/fast-downward-24.06.1/builds/release/bin/translate/pddl_to_prolog.py", line\n153, in translate_facts\n  translate_typed_object(prog, obj, type_dict)\n File\n"/mnt/c/Users/pablo/Desktop/TSI_2024-25/P3/Soluciones/FD/fast-downward-\n24.06.1/builds/release/bin/translate/pddl_to_prolog.py", line 146, in translate_typed_object\n  supertypes = type_dict[obj.type_name].supertype_names\nKeyError: 'recurso'\n'\ntranslate exit code: 30
```

Driver aborting after translate

Da un error muy verboso y poco informativo. Aunque, al final de todo, sí nos da una pista con el término que origina el error.

Errores típicos y mensajes asociados (5)

- En el fichero de dominio, falta un espacio a la hora de definir un tipo. Por ejemplo, tenemos **Recurso- Ubicable** en lugar de **Recurso - Ubicable**:
 - FF: predicate EN is declared to use unknown or empty type UBICABLE



Da un error muy escueto y poco informativo. Pero hace referencia a que hay un problema con el tipo Ubicable. Vemos que, por ejemplo, mientras FD hace énfasis en que el problema está en “recurso”, FF sugiere que el problema está en “ubicable” (una vez procesa el predicado “en”).

Errores típicos y mensajes asociados (6)

- En el fichero de problema, falta un espacio a la hora de definir un objeto. Por ejemplo, tenemos `Enano1 Hobbit1 Enano2- Personaje` en lugar de `Enano1 Hobbit1 Enano2 - Personaje`:

- FD:

Search stopped without finding a solution.

← No da error! Sencillamente, dice que no encuentra plan!

- FF:

predicate PERSONAJEES is declared to use unknown or empty type PERSONAJE

↗
Da un error asociado con el predicado en que se asigna tipo al personaje ("personajees").

Errores típicos y mensajes asociados (7)

- En el fichero de problema, creamos un objeto de un tipo inexistente. Por ejemplo, tenemos `Enano1 Hobbit1 Enano2 - Personaje2` (y `Personaje2` no es un tipo que exista):

- FD:

```
Generating Datalog program... b'Traceback (most recent call last):\n File\n"/mnt/c/Users/Usuario/Documents/_UGR_/Docencia/TSI_2024-25/P3/Soluciones/FD/fast-downward-24.06.1/builds/release/bin/translate/translate.py", line 747, in <module>\n  main()\n File\n"/mnt/c/Users/Usuario/Documents/_UGR_/Docencia/TSI_2024-25/P3/Soluciones/FD/fast-downward-24.06.1/builds/release/bin/translate/translate.py", line 718, in main\n  sas_task = pddl_to_sas(task)\n File\n"/mnt/c/Users/Usuario/Documents/_UGR_/Docencia/TSI_2024-25/P3/Soluciones/FD/fast-downward-24.06.1/builds/release/bin/translate/translate.py", line 547, in pddl_to_sas\n  reachable_action_params) =\n  instantiate.explore(task)\n File\n"/mnt/c/Users/Usuario/Documents/_UGR_/Docencia/TSI_2024-25/P3/Soluciones/FD/fast-downward-24.06.1/builds/release/bin/translate/instantiate.py", line 116, in\n  explore\n  prog = pddl_to_prolog.translate(task)\n File\n"/mnt/c/Users/Usuario/Documents/_UGR_/Docencia/TSI_2024-25/P3/Soluciones/FD/fast-downward-24.06.1/builds/release/bin/translate/pddl_to_prolog.py", line 167, in translate\n  translate_facts(prog,\n  task)\n File\n"/mnt/c/Users/Usuario/Documents/_UGR_/Docencia/TSI_2024-25/P3/Soluciones/FD/fast-downward-24.06.1/builds/release/bin/translate/pddl_to_prolog.py", line 153, in translate_facts\n  translate_typed_object(prog, obj, type_dict)\n File\n"/mnt/c/Users/Usuario/Documents/_UGR_/Docencia/TSI_2024-25/P3/Soluciones/FD/fast-downward-24.06.1/builds/release/bin/translate/pddl_to_prolog.py", line 146, in translate_typed_object\n  supertypes\n  = type_dict[obj.type_name].supertype_names\nKeyError: 'personaje2'\n'\ntranslate exit code: 30
```

Driver aborting after translate.

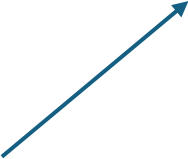
Indica al final de todo el origen del problema (término “personaje2”).

Errores típicos y mensajes asociados (7)

- En el fichero de problema, creamos un objeto de un tipo inexistente. Por ejemplo, tenemos `Enano1 Hobbit1 Enano2 – Personaje2` (y `Personaje2` no es un tipo que exista):

- FF:

predicate PERSONAJEES is declared to use unknown or empty type PERSONAJE



Da un error asociado con el predicado en que se asigna tipo al personaje (“personajees”).

Errores típicos y mensajes asociados (8)

- Utilizamos **impl** en los efectos, en lugar de en las precondiciones de una acción:

- FD:

Parsing domain

- >Parsing 5. axiom/action entry
- >Parsing 5. action
- >Parsing action 'recogerobjeto'
- >Parsing effect
- >Parsing effect
- >Parsing literal

Undeclared predicate

Got: **impl**

El mensaje, nuevamente, no es particularmente informativo. Es decir, no es que nos diga, explícitamente, que en los efectos no se puede emplear **impl**. Pero sí nos indica que hay un problema con el **impl** dentro de la acción recogerobjeto.

- FF:

ff: parsing domain file

dominio2.pddl: syntax error in line 179, 'impl':

Nos indica que hay un problema con un **impl** en la línea 179.

Errores típicos y mensajes asociados (9)

"This configuration does not support axioms!"

Terminating.

Tried to use unsupported feature."

- Esto ocurre en FD, en ocasiones, cuando empleamos forall/exists y ejecutamos con seq-opt-lmcut
 - El motivo es que FD traduce PDDL a un formato intermedio denominado SAS+. En ese proceso de traducción, algunos cuantificadores se transforman en axiomas (también conocidos como *derived predicates*).
 - Dichos axiomas no son soportados por la búsqueda óptima (seq-opt-lmcut).
 - LM-cut actualmente no está implementada para manejar adecuadamente axiomas.
 - Por tanto, aunque la traducción acepte tu dominio con forall/exists, la fase de búsqueda de seq-opt-lmcut detecta las reglas derivadas y aborta con "This configuration does not support axioms!"
- Solución: emplear directivas de ejecución que no fuercen optimalidad, como seq-sat-lama-2011 (con un determinado search-time-limit). Esto proporcionará un plan válido que, progresivamente, intentará ir optimizando.

Errores típicos y mensajes asociados (9)

“This configuration does not support axioms!”

Terminating.

Tried to use unsupported feature.”

- ¿Qué son exactamente los axiomas (*derived predicates*)?
 - Predicados cuya verdad (True/False) no cambia directamente (por medio de los efectos de las acciones, por ejemplo) sino que es inferida a partir de otros predicados/hechos.

- Ejemplo:

```
(:derived (train-usable ?t - train)
  (and
    (train-has-guard ?t)
    (train-has-driver ?t)
  )
)
```

“A train is only usable if it has a guard and a driver”

- Véase
 - <https://planning.wiki/ref/pddl22/domain>

Errores típicos y mensajes asociados (9)

"This configuration does not support axioms!"

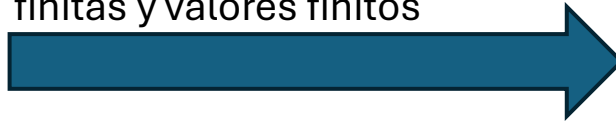
Terminating.

Tried to use unsupported feature."

- ¿De qué forma un forall se puede convertir en un axioma (*derived predicate*)?

```
(forall (?b - block)
  (clear ?b)
)
```

Traducción a SAS+ con variables
finitas y valores finitos



axioma:

```
context: (clear A) AND
         (clear B) AND
         (clear C)
```

```
implies: new-axiom-all-clear
```

"Universal conditions in preconditions, effect conditions and the goal are internally compiled into axioms by the planner"

Errores típicos y mensajes asociados (9)

“This configuration does not support axioms!

Terminating.

Tried to use unsupported feature.”

- ¿Por qué FD convierte, en ocasiones, cuantificadores (forall, exists) en axiomas (*derived predicates*)?
 - Porque esto simplifica la estructura lógica del problema.
 - Para la mayoría de heurísticas y algoritmos de búsqueda, es mucho más sencillo trabajar sobre un dominio *ground* (completamente instanciado).
 - Esto permite separar “evaluación de cuantificadores” y “algoritmo de búsqueda”.
- Véase
 - <https://www.fast-downward.org/24.06/documentation/pddl-support/>
 - <https://www.fast-downward.org/HEAD/documentation/translator-output-format/>

Errores típicos y mensajes asociados (9)

"This configuration does not support axioms!"

Terminating.

Tried to use unsupported feature."

- ¿Por qué seq-opt-lmcut no soporta axiomas?
 - Porque los axiomas (*derived predicates*) introducen ciertas complicaciones técnicas, y la implementación actual de LM-cut en FD prima eficiencia y optimalidad.
 - Por ejemplo, un axioma puede depender de otros axiomas o de efectos de acciones, formando cadenas de inferencia → esto implicaría manejar reglas lógicas recursivas.
- Véase
 - https://www.fast-downward.org/HEAD/documentation/search/Evaluator/#landmark-cut_heuristic
 - Liang, Yichao, et al. "VisualPredicator: Learning abstract world models with neuro-symbolic predicates for robot planning." (ICLR'25) <https://arxiv.org/pdf/2410.23156> ("*Our use of derived predicates is not out-of-box compatible with relaxed planning heuristics, such as LM-cut*")
 - Thiébaux, Sylvie, Jörg Hoffmann, and Bernhard Nebel. "In defense of PDDL axioms." *Artificial Intelligence* 168.1-2 (2005): 38-69. https://icaps03.icaps-conference.org/satellite_events/documents/WS/WS1/04/Sylvie.pdf ("*Axioms bring (much needed) expressive power, but [...] extending a planner to explicitly deal with axioms may lead to much better performance than using a compilation scheme with the original version of the planner*")

¿PDDL es un lenguaje de programación interpretado o compilado?

¿PDDL es un lenguaje de programación interpretado o compilado?

- Java:
 - .java se compila a código intermedio (*bytecode*): generando .class
 - Este *bytecode* es independiente de la plataforma, y puede ejecutarse en cualquier sistema que tenga una máquina virtual (JVM) compatible.
 - La JVM interpreta o compila el *bytecode* para ejecutarlo en la máquina específica.

"Java is both a compiled and interpreted programming language. Because it employs both compilation and interpretation to run code, it is known as a "compiler-interpreter language." In order for the Java Virtual Machine to understand the Java code, it must first be compiled into an intermediate format called bytecode (JVM). The bytecode is subsequently interpreted by the JVM and transformed into hardware-compatible machine code. "

<https://www.geeksforgeeks.org/why-java-is-called-a-compiler-interpreter-language/>

¿PDDL es un lenguaje de programación interpretado o compilado?

- MiniZinc:
 - Compila el fichero .mzn a un lenguaje intermedio llamado FlatZinc.
 - Este proceso de compilación traduce el modelo y los datos de entrada en una representación más simple y estandarizada que puede ser procesada por diversos *solvers*.
 - Este enfoque permite que MiniZinc sea independiente del solver, facilitando la comparación y el uso de diferentes *solvers* para un mismo modelo.
 - Luego, MiniZinc ejecuta el modelo usando un *solver* externo (como Gecode).

Véase <https://docs.minizinc.dev/en/stable/flattening.html> o https://cpmpy.readthedocs.io/en/docs_api_quickstart_page/api/solvers/minizinc.html

¿PDDL es un lenguaje de programación interpretado o compilado?

- PDDL:
 - Algunos planificadores pueden realizar una **traducción intermedia** o **compilación** de las descripciones PDDL a otras representaciones internas para optimizar el proceso de planificación, pero esto **depende del planificador específico utilizado**.
 - Por ejemplo, Fast Downward convierte código .pddl a un formato intermedio .sas
 - Este proceso transforma la descripción del dominio y del problema en PDDL a una representación interna conocida como **SAS+** (State-Action-State Plus).
 - Traducción (PDDL → SAS+) y búsqueda (en SAS+)
 - Fast Forward, por ejemplo, podría ser un ejemplo de planificador interpretado
 - Procesa PDDL sin producir una representación intermedia.

Véase <https://icaps22.icaps-conference.org/workshops/DAPSPAC/papers/zhang2022using.pdf>